**LAB 1: PREDICTING CUSTOMER PURCHASE**

## Estimated time

20-30 minutes

## Level of difficulty

Intermediate

## Objectives

- Train a Decision Tree classifier using the *customer purchase dataset*.
- Predict whether a customer will buy a product based on age, income, and marital status.
- Perform a train/test split and evaluate the model's accuracy.

## Scenario

You are working as a junior data analyst for an e-commerce company.
Your task is to build a simple machine learning model that predicts whether a customer will purchase a product based on:

- Age
- Income level (monthly)
- Marital status

You will use a Decision Tree Classifier from scikit-learn.

**Step 1: Import and Load the Dataset**

Load customer_data.csv using pandas.
Display the dataset.

**Step 2: Prepare the Data**

1. Convert **marital_status** and **purchase** to numerical values.
2. Split data into:
    - X → features (age, income, marital_status)
    - y → labels (purchase)

**Step 3: Create the Model**

Create a DecisionTreeClassifier() object.

**Step 4: Train the Model**

Train using:

model.fit(X, y)

**Step 5: Make Predictions**

Predict for the following new customer:

| Age | Income | Married |
|---|---|---|
| 30 | 5000 | 0 |

**Step 6: Evaluate the Model**

Use train_test_split() and accuracy_score() to calculate accuracy.

## LAB 2: PREDICTING EMPLOYEE SALARY

**Estimated Time**

20–30 minutes

**Level of Difficulty**

Intermediate

---

**Objectives**

- Understand **multiple linear regression**
- Handle **missing values** in a dataset
- Convert **categorical text data** into numerical values
- Train a regression model using scikit-learn
- Predict employee salaries based on multiple features

---

**Scenario**

You are working as a **junior data analyst** for a recruitment company.
The company wants to **predict employee salaries** based on candidate information collected during the hiring process.

The available features are:

- Years of experience
- Test score (out of 10)
- Interview score (out of 10)

Your task is to clean the dataset, train a **Linear Regression model**, and make salary predictions for new candidates.

---

**Lab Tasks / Questions**

**Step 1: Load the Dataset**

- Load the dataset hiring.csv using pandas
- Display the original dataset

---

**Step 2: Handle Missing Experience Values**

- Replace missing values in the experience column with "zero"
- Convert experience values written in words (e.g. "two", "five") into numeric values

- Display the processed dataset

---

**Step 3: Handle Missing Test Scores**

- Calculate the **mean test score**
- Round it down using math.floor()
- Replace missing values in test_score(out of 10) with the calculated value
- Display the updated dataset

---

**Step 4: Train the Linear Regression Model**

- Create a LinearRegression() model
- Train the model using:
    - **Features (X):**
        - experience
        - test_score(out of 10)
        - interview_score(out of 10)
    - **Target (y):**
        - salary($)

---

**Step 5: Make Salary Predictions**

Use the trained model to predict salaries for the following candidates:

1. Candidate A
    - Experience: 2 years
    - Test Score: 9
    - Interview Score: 6
2. Candidate B
    - Experience: 12 years
    - Test Score: 10
    - Interview Score: 10

## LAB 3: FLOWER CLASSIFICATION USING RANDOM FOREST

**Estimated Time**

20–30 minutes

**Level of Difficulty**

Intermediate

---

**Objectives**

- Understand **ensemble learning** using Random Forest
- Train a **Random Forest Classifier** on a labeled dataset
- Perform **train–test splitting**
- Compare model performance using different numbers of trees
- Evaluate model accuracy

---

**Scenario**

You are working as a **junior machine learning engineer** in a research lab.
Your task is to build a model that **classifies iris flowers into their species** based on flower measurements.

You will use the **Iris dataset**, which contains:

- Sepal length
- Sepal width
- Petal length
- Petal width

The target variable represents the **flower species**.

---

**Lab Tasks / Questions**

**Step 1: Load the Dataset**

- Load the Iris dataset using load_iris() from sklearn.datasets
- Print the dataset keys to understand its structure

---

**Step 2: Create a DataFrame**

- Convert the dataset into a pandas DataFrame
- Add the target labels as a new column called target

- Display the first few rows of the dataset

---

**Step 3: Define Features and Target**

- Separate the dataset into:
    - **Features (X)** → all columns except target
    - **Target (y)** → target

---

**Step 4: Split the Dataset**

- Split the data into training and testing sets
- Use:
    - test_size = 0.2
    - random_state = 42

---

**Step 5: Train a Random Forest Classifier (Default Parameters)**

- Create a RandomForestClassifier using default parameters
- Train the model on the training data
- Evaluate and print the model accuracy on the test data

---

**Step 6: Train a Random Forest Classifier with 40 Trees**

- Create another Random Forest model with:
    - n_estimators = 40
- Train the model on the same training data
- Evaluate and print the accuracy

---

**Step 7: Compare Results**

- Compare the accuracy of:
    - Default Random Forest
    - Random Forest with 40 trees
- Discuss how the number of trees affects model performance

## LAB 4: FLOWER SEGMENTATION

**Estimated Time**

30 minutes

**Level of Difficulty**

Intermediate

---

**Objectives**

- Understand **unsupervised learning** using K-Means clustering
- Apply **feature scaling** before clustering
- Use the **Elbow Method** to determine the optimal number of clusters
- Visualize clusters and centroids using matplotlib
- Perform clustering on the **Iris dataset** without using class labels

**Scenario**

You are a junior data analyst working in a data science team.
Your task is to **group flowers based on their physical characteristics** without knowing their species in advance.

You will use **K-Means clustering** to discover natural groupings in the Iris dataset based on:

- Petal length
- Petal width

---

**Lab Tasks / Questions**

**Step 1: Load the Dataset**

- Load the **Iris dataset** using load_iris() from sklearn.datasets
- Convert the dataset into a pandas DataFrame
- Display the first 5 rows of the dataset

---

**Step 2: Select Features**

- Select the following features:
    - petal length (cm)
    - petal width (cm)
- Create a new DataFrame containing only these features

---

**Step 3: Visualize the Data (Before Clustering)**

- Create a **scatter plot** of:
    - X-axis: Petal Length
    - Y-axis: Petal Width
- Observe how the data points are distributed

---

**Step 4: Feature Scaling**

- Apply **Min-Max Scaling** to the selected features
- Explain why feature scaling is important for K-Means clustering

---

**Step 5: Apply the Elbow Method**

- Use K-Means clustering for **K = 1 to 10**
- Calculate the **Sum of Squared Errors (SSE)** for each K
- Plot the Elbow graph
- Determine the optimal number of clusters

---

**Step 6: Train K-Means Model**

- Train a **K-Means model with K = 3**
- Predict cluster labels for the dataset

---

**Step 7: Add Cluster Labels**

- Add a new column called cluster to the DataFrame
- Display the updated DataFrame

---

**Step 8: Visualize the Clusters**

- Plot each cluster using a different color
- Add labels and legend to the plot
- Interpret the clustering result

---

**Step 9: Plot Cluster Centroids**

- Extract cluster centroids from the trained model

- Plot the centroids on top of the clustered data
- Mark centroids using a star (*) symbol

---

**Step 10: Compare with Actual Labels (Optional)**

- Display the actual Iris target labels
- Compare clustering results with real species labels
- Discuss whether K-Means successfully grouped the flowers

---

LAB 5: BUILDING CHATBOT

## Estimated time

20-30 minutes

## Level of difficulty

Intermediate

## Objectives

- build a small chatbot that sends user prompts to the **DeepSeek model on OpenRouter** and prints the model's response.
- integrate a third-party LLM (Large Language Model) into a Python application using the **OpenRouter REST API** and the **OpenAI Python SDK**

## Scenario

**Step 1: Template for Connecting to DeepSeek**

Refer to the following template:

```
from openai import OpenAI
client = OpenAI(
  base_url="https://openrouter.ai/api/v1",
  api_key="<OPENROUTER_API_KEY>",
)
completion = client.chat.completions.create(
  extra_headers={
    "HTTP-Referer": "<YOUR_SITE_URL>",
    "X-Title": "<YOUR_SITE_NAME>",
  },
  extra_body={},
  model="<YOUR MODEL>",
  messages=[
     {
       "role": "user",
       "content": "What is the meaning of life?"
     }
  ]
)

print(completion.choices[0].message.content)
```

**Step 2: Create a Chat Function** that sends a prompt to Chatgpt and returns the response.

**Step 3: Create a Chat Loop** used to allow real-time interaction.

LAB 6: CHATBOT MEMORY

## Estimated time

30 minutes

## Level of difficulty

Intermediate

## Objectives

- Create a command-line chatbot that remembers the conversation using the OpenAI SDK with the OpenRouter API

In your first lab:

1. Import the OpenAI class from the openai module. This class lets you send messages to the chatbot.
2. Create a client to connect to OpenRouter.Use your actual OpenRouter API key (keep it secret!). This sets up the connection.
3. **Create a list to store your conversation.** You'll use this to make the bot remember everything.
4. **Define a function called chat_with_DS(prompt)**. Inside the function:
   • Add the user's message to message_history
   • Send the full history to the chatbot
   • Add the chatbot's reply to the history
   • Return the reply
5. **Write the main chat loop.**
   • Let the user keep typing questions
   • End the loop if they type "exit", "quit" or "bye"
   • Print the chatbot's response

**Run the code and say "Hello I am [*Your name*]". Then ask a follow-up like "What's my name?" Does it remember?**